



Phytosend

Progetto TWeb AA 25-26



Rudisi Salvatore



Front-end: caratteristiche di base

1. Tecnologie Utilizzate

- Spring Boot 3.5.8 (Java 25)
- React 19 (TypeScript)
- PostgreSQL
- Docker

Front-end: caratteristiche di base

2. Single-Page-Application realizzata con React.

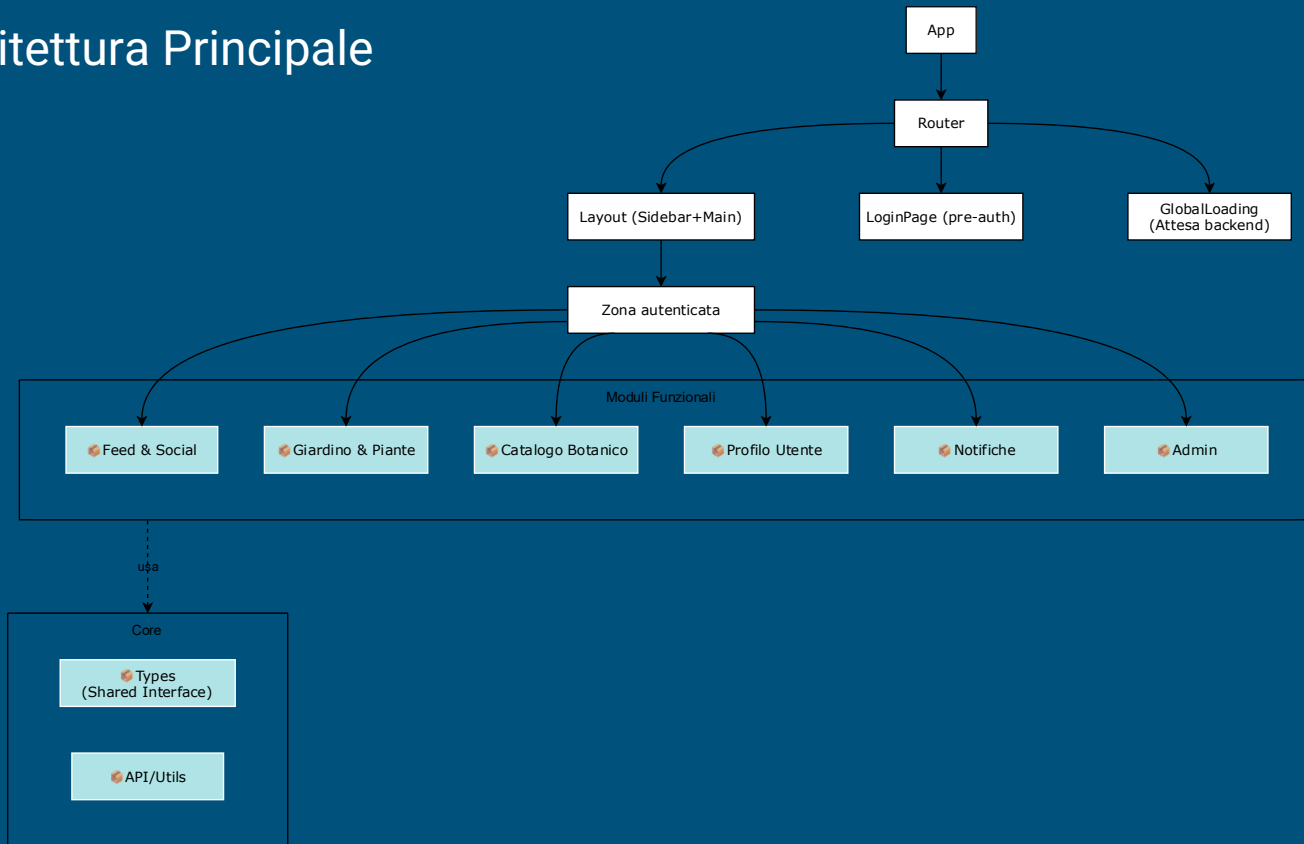
Il frontend carica un unico `index.html` e aggiorna dinamicamente la UI tramite routing client-side (`react-router-dom`). Il backend è un'API REST separata.

Pagine (Client-Side Routes):

- **Home** (/) – Feed sociale con post, like, commenti
- **Ricerca** (/search) – Catalogo botanico
- **Dettaglio Pianta** (/plant/:id) – Scheda botanica
- **Profilo** (/profile, /profile/:userId) – Profilo utente
- **Il Mio Giardino** (/my-garden, /garden/:userId) – Gestione piante e eventi di cura
- **Post Salvati** (/saved-posts) – Archivio post salvati
- **Notifiche** (/notifiche) – Cronologia notifiche
- **Admin** (/admin) – Pannello amministrativo (solo ADMIN)
- **Login** (pre-auth) – Autenticazione e registrazione

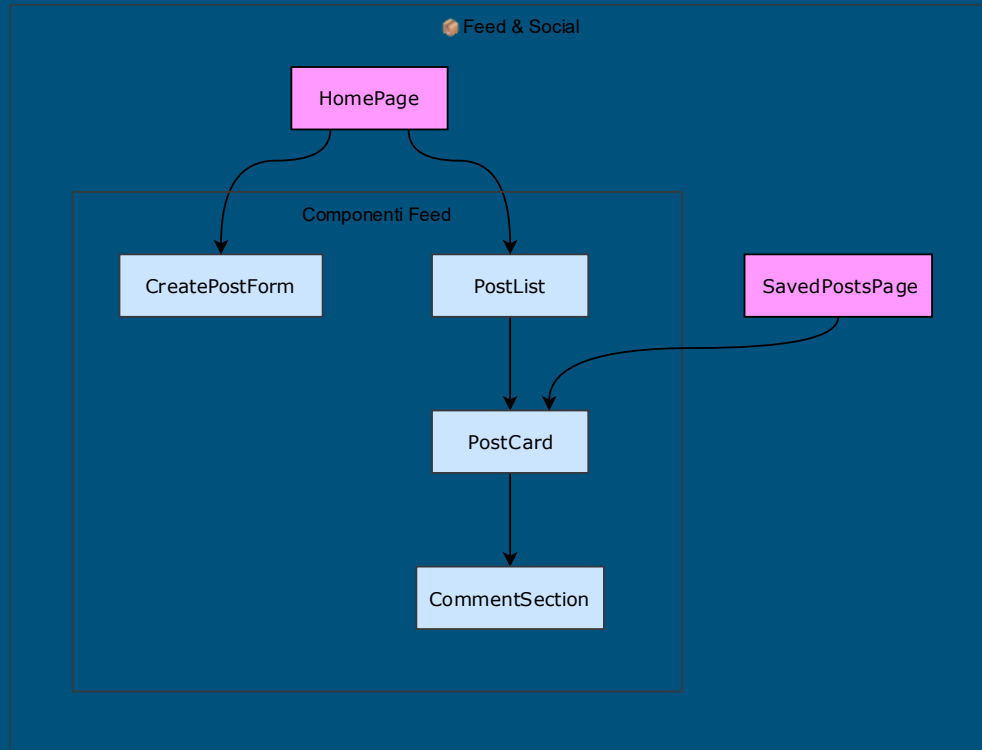
Front-end: componenti UI

1. Architettura Principale



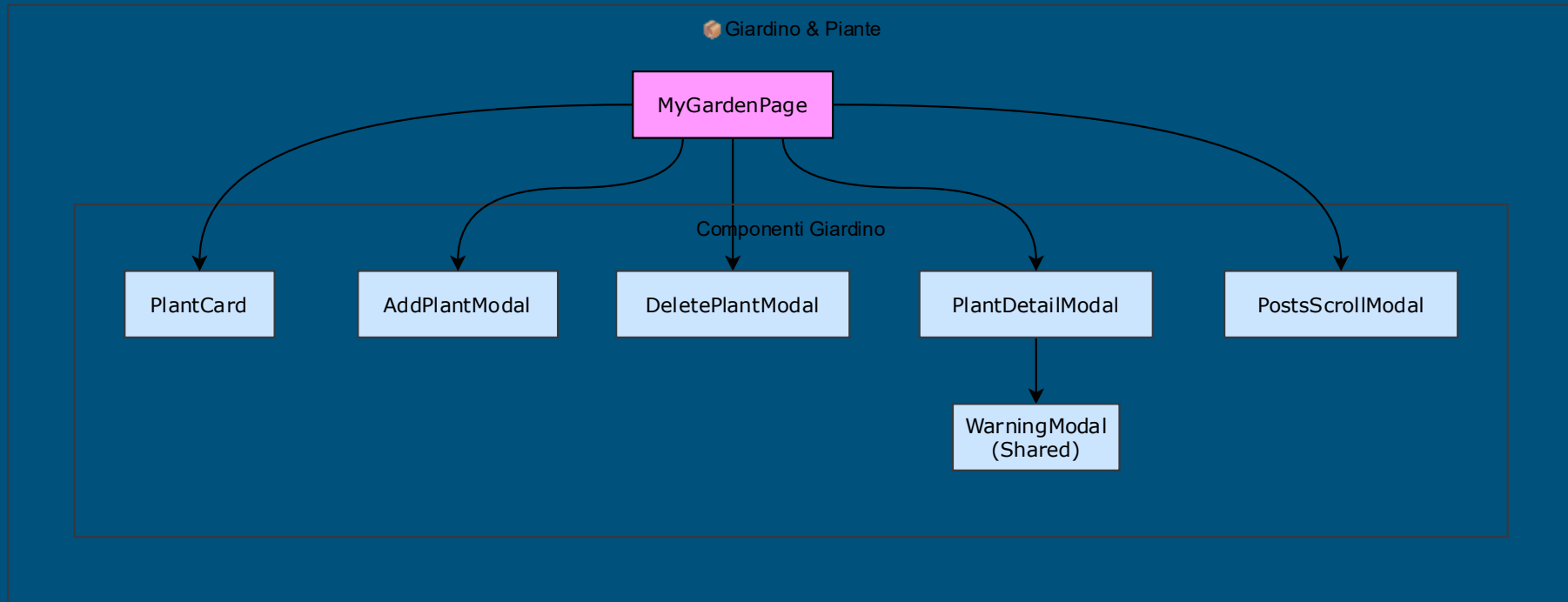
Front-end: componenti UI

2. Modulo Feed & Social



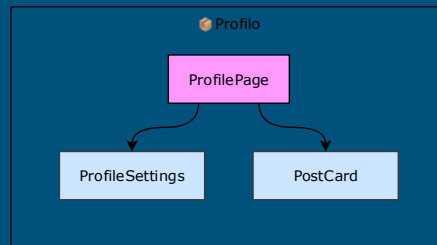
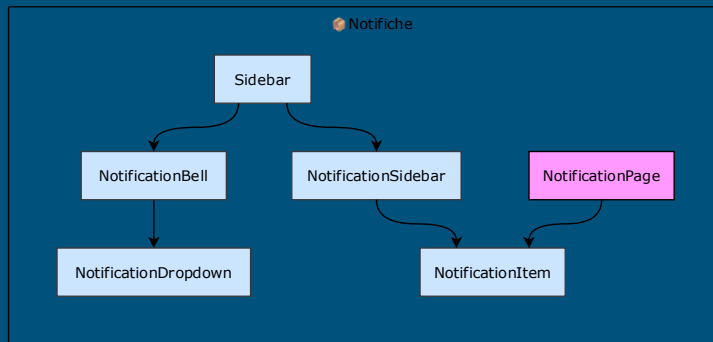
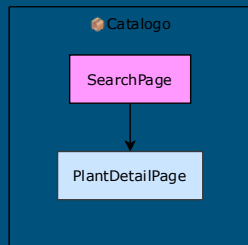
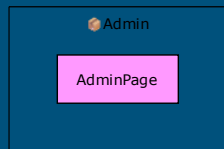
Front-end: componenti UI

3. Modulo Giardino & Piante

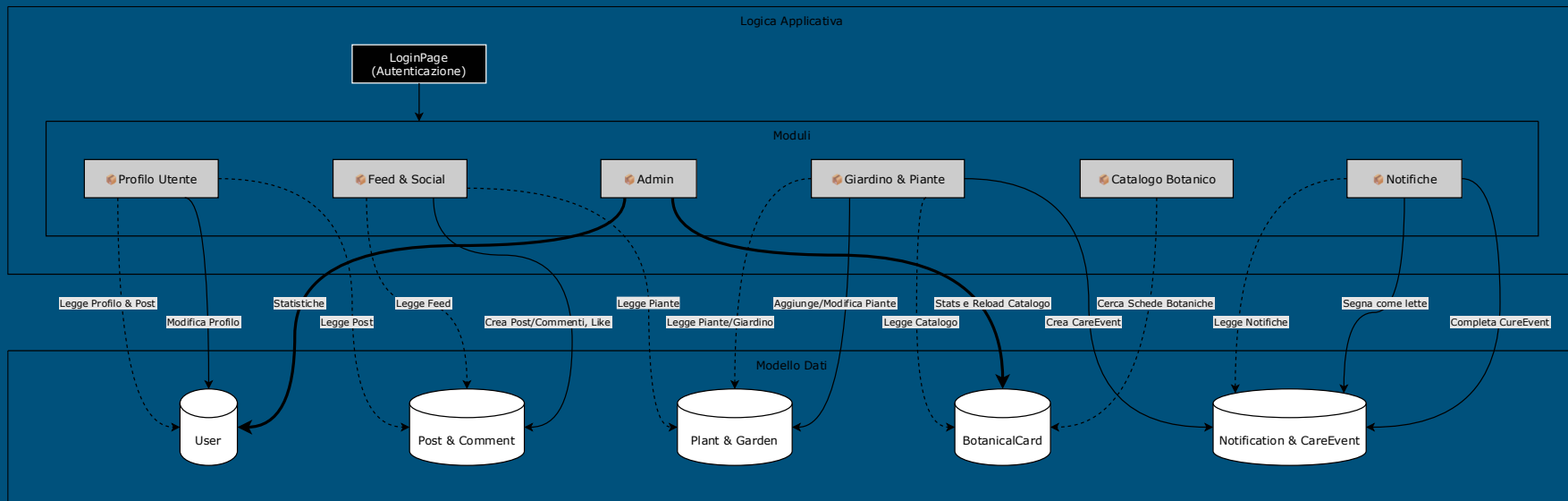


Front-end: componenti UI

4. Moduli Profilo, Catalogo, Notifiche, Admin



Front-end: UI e modello dati



Back-End: struttura



Back-End: routes

AuthController (/api/auth):

- `POST /login` — Login utente (body: email, password)
- `POST /register` — Registrazione (body: dati utente)

UserController (/api/utenti):

- `GET /` — Lista tutti gli utenti
- `GET /{id}` — Dettaglio utente per ID
- `POST /` — Creazione utente
- `PUT /{id}` — Aggiornamento dati utente
- `POST /{userId}/piante` — Aggiunta pianta al giardino
- `PUT /{userId}/piante/{plantId}/name` — Rinomina pianta
- `DELETE /{userId}/piante/{plantId}` — Eliminazione pianta
- `PUT /{userId}/piante/{plantId}/dead` — Segna pianta morta
- `POST /{userId}/piante/{plantId}/care-events` — Crea evento di cura

Back-End: routes

SocialController (/api/social):

- `GET /posts` — Feed post (paginato)
- `GET /posts/user/{userId}` — Post di un utente
- `GET /posts/plant/{plantId}` — Post di una pianta
- `GET /posts/{postId}/likes` — Utenti che hanno messo like
- `POST /posts` — Creazione post
- `DELETE /posts/{postId}` — Eliminazione post
- `GET /posts/{postId}/commenti` — Commenti di un post
- `POST /posts/{postId}/commenti` — Aggiunta commento
- `DELETE /posts/{postId}/commenti/{commentId}` — Eliminazione commento
- `POST /posts/{postId}/like` — Toggle like su post
- `POST /commenti/{commentId}/like` — Toggle like su commento
- `POST /posts/{postId}/save` — Toggle salvataggio post
- `GET /posts/saved` — Post salvati

Back-End: routes

PlantController (/api/utenti):

- `GET /{utenteld}/piante` — Lista piante di un utente

GardenController (/api/gardens):

- `GET /user/{userId}` — Giardino di un utente
- `POST /user/{userId}` — Creazione giardino
- `PUT /{gardenId}` — Aggiornamento giardino

CatalogController (/api/catalogo):

- `GET /user/{userId}` — Giardino di un utente
- `POST /user/{userId}` — Creazione giardino
- `PUT /{gardenId}` — Aggiornamento giardino

Back-End: routes

NotificationController (/api/notifications):

- `GET /` — Tutte le notifiche (paginate)
- `GET /count` — Conteggio non lette
- `GET /recent` — Ultime notifiche
- `PUT /{notificationId}/read` — Segna come letta
- `PUT /read-all` — Segna tutte come lette
- `POST /care-events/{eventId}/complete` — Completa evento di cura

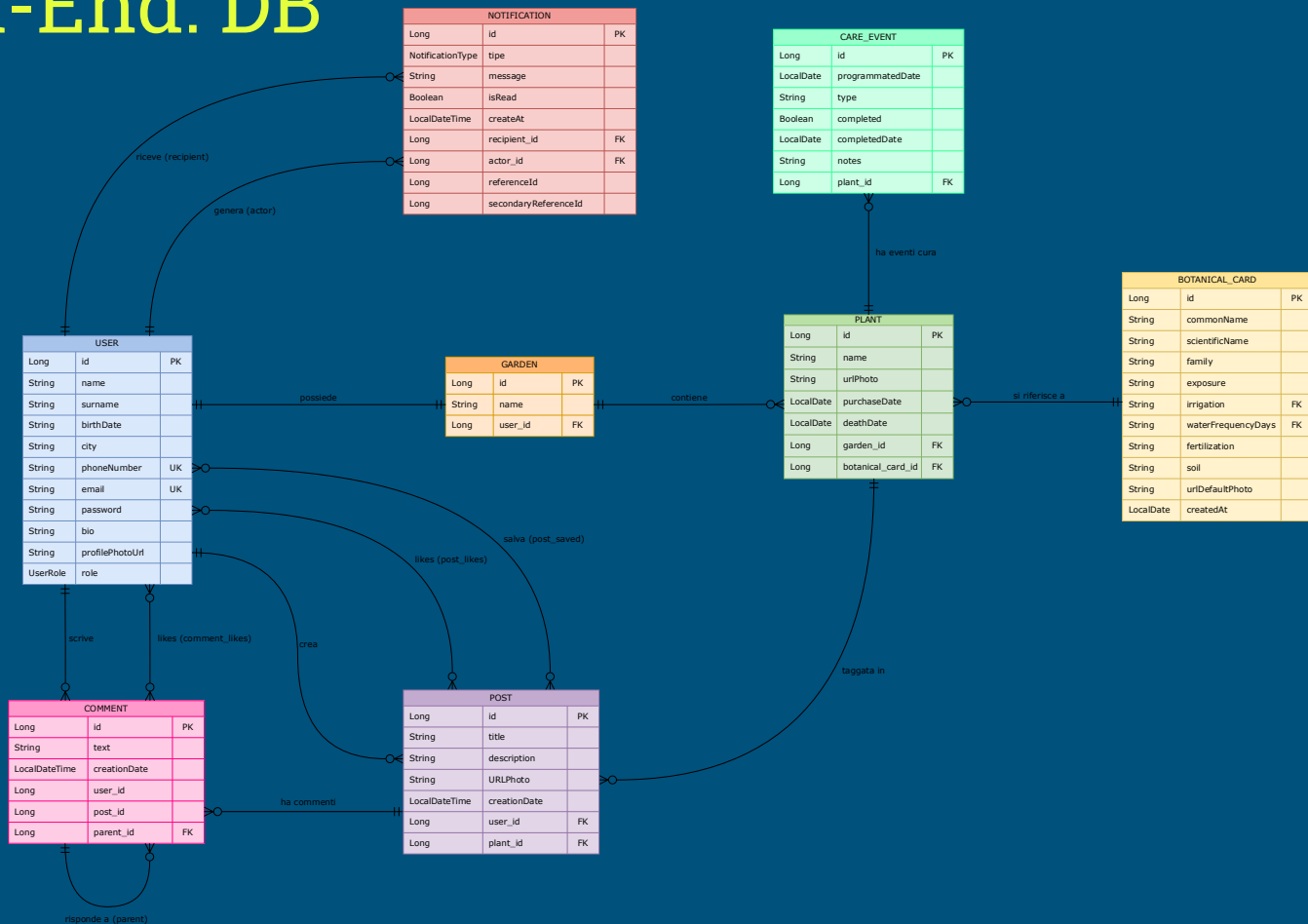
FileUploadController (/api/upload):

- `POST /profile-photo` — Upload foto profilo
- `DELETE /profile-photo` — Eliminazione foto profilo

AdminController (/api/admin):

- `GET /stats` — Statistiche sistema
- `POST /reload-catalog` — Ricarica catalogo botanico

Back-End: DB



Note Particolari

- **Notifiche Automatiche Cura Piante:** Il CareEventScheduler genera automaticamente eventi di irrigazione basandosi sulla frequenza nella scheda botanica, con notifiche push all'utente.
- **Autenticazione Stateless JWT:** JwtAuthenticationFilter personalizzato integrato con Spring Security; nessuna sessione server-side.
- **Monitoring Produzione:** Spring Actuator + Micrometer Prometheus per metriche real-time del backend.